READING MATERIAL

ContextTrace

Cloudflare Workers, Workers AI embeddings and vector search, learned from the code

ContextTrace turns the issues, pull requests and comments of a public GitHub repository into searchable evidence. Every search result is a passage with a verified permalink back to the comment it came from. This document explains the technology the project uses and points at the exact files and lines to read for three topics: Cloudflare Workers, embeddings with Workers AI, and the vectorization pipeline that ends in a pgvector search.

Source read: branch `main` at commit `cfd6f44` (merged 2026-10-23)

Repository: github.com/pudasainimiraj/ContextTrace

Live service: contexttrace.contexttrace.workers.dev and pudasainimiraj.github.io/ContextTrace

Document date: 2026-10-24

Contents

1. How to use this document

2. The system on one page

2.1 The pipeline

2.2 From a URL to an evidence card

2.3 The numbers that shape the design

3. Technology stack

4. Track A: Cloudflare Workers

A.1 The Worker configuration: wrangler.jsonc

A.2 One module, three entry points: fetch, scheduled, queue

A.3 The typed environment: bindings, vars and secrets

A.4 Cloudflare Queues: producer and consumer

A.5 Cron Triggers

A.6 Living inside the free plan CPU limit

A.7 Rate limiting, static assets, CORS, Turnstile

A.8 Developing and testing a Worker locally

A.9 Deploying

5. Track B: Embeddings with Workers AI

B.1 What the model does

B.2 The REST client

B.3 Reading the response

B.4 Error mapping

B.5 Metering, neurons and cost

B.6 Validating a vector

B.7 The embed job

B.8 Embedding the search query

B.9 Testing without the model

B.10 Counting tokens the way the model does

6. Track C: Vectorization, from GitHub text to a vector search

C.1 The job chain

C.2 Extract

C.3 Normalize

C.4 Chunk

C.5 Passages, deterministic ids and content hashes

C.6 Writing passages: sync_passages and two-phase indexing

C.7 Storing vectors: the pgvector table and the HNSW index

C.8 Searching vectors: cosine on the HNSW index

C.9 The keyword baseline

C.10 Deciding what to show

C.11 What the evaluation showed

7. Reading paths and exercises

8. Glossary and references

Appendix: File index

1. How to use this document

This is a study guide, not a manual. Each section names a file and a line range in the ContextTrace repository. Open the file next to the section and read the two together. The code excerpts in this document are shortened. A line with `…` marks a cut. Line numbers refer to branch `main` at commit `cfd6f44`; they drift when the code changes, so trust the function names over the numbers.

The guide has three tracks. Each track can be read on its own.

- **Track A, Cloudflare Workers.** How one Worker module serves HTTP, consumes a queue and runs on a cron, and how the project stays inside the free plan.
- **Track B, Embeddings with Workers AI.** How a text becomes a 384-dimensional vector, what the REST call looks like, how it is metered, validated, tested and paid for.
- **Track C, Vectorization.** How GitHub Markdown becomes token-bounded passages with stable ids, how vectors are stored in Postgres with pgvector, and how a search ranks them.

Section 2 gives the whole picture first. Section 7 gives a reading order and exercises. Section 8 holds the glossary and the external references.

> **What ContextTrace is.** A developer registers a public GitHub repository. The service fetches its issues, pull requests, issue comments and review comments, splits every opening post and comment into passages of at most 256 model tokens, embeds each passage with the `@cf/baai/bge-small-en-v1.5` model on Workers AI, stores the vectors in Supabase Postgres with pgvector, and answers questions by cosine similarity. The response is a set of evidence cards, each with a permalink that the server verified. Everything runs on free tiers; the service meters itself and pauses rather than fails.

2. The system on one page

2.1 The pipeline

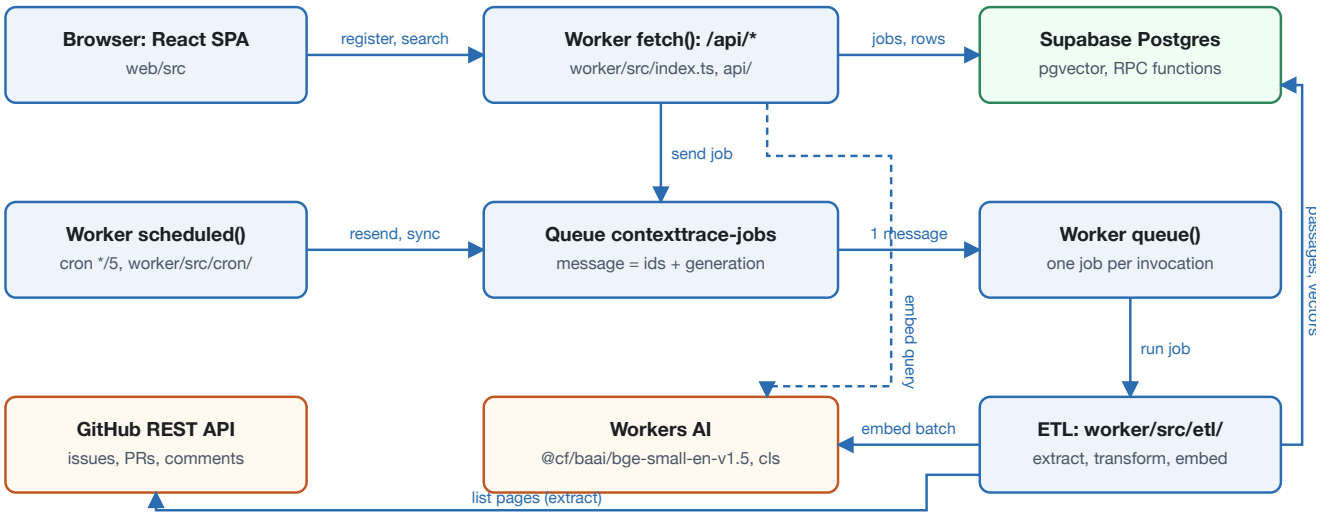


Figure 1. The Worker is one module with three handlers. The queue carries 76-byte messages; the job rows in Postgres are the source of truth. Workers AI is called by the consumer to embed passages and by the HTTP handler to embed a search query. The cron handler also reads and writes Postgres directly.

2.2 From a URL to an evidence card

1. **Registration.** `POST /api/repositories` validates the URL, checks Turnstile and admissions, reserves a registration in the budget, looks the repository up on GitHub, upserts the row keyed by GitHub's numeric id, and queues one `fetch_<stream>` job per stream (issues, issue comments, review comments).
2. **Head pass and backfill.** The first run of each stream lists the newest two pages. Backfill jobs continue three pages per run with a 10 s delay, up to 5,000 items per stream.
3. **Transform.** `transform_passages` groups a thread's rows into a document, normalizes the Markdown, splits each unit into passages of at most 256 tokens, computes deterministic ids and content hashes, and writes them through the `sync_passages` SQL function.
4. **Embed.** `embed_passages` takes pending passages in batches sorted by length, reserves the inference budget, calls Workers AI with `cls` pooling, validates every vector and stores it in `passage_embeddings`. The repository becomes `searchable_partial` after the first batch.
5. **Search.** `POST /api/search` embeds the query on the server, ranks the repository's active vectors by cosine similarity on the HNSW index, and decides between a high-confidence semantic answer, a keyword fallback, a weak semantic answer with a warning, or "no supporting discussion".
6. **Sync and rescan.** The cron queues a sync per complete stream every hour and a low-priority rescan weekly, which retires rows GitHub no longer lists.

2.3 The numbers that shape the design

Constant	Value	Where	Why it matters
Embedding dimensions	384	`etl/embeddings.ts:10`, table CHECK	Small vectors: about 1.5 kB of float4 per row, 4.4 kB with the index.
Pooling	cls	`etl/embeddings.ts:9`	The model's native pooling. Workers AI defaults to `mean`; the two are not comparable.
Model sequence length	512	`etl/embeddings.ts:12`	A passage above it is never sent; it is counted as oversized.

Tokens per passage	256	`etl/chunk.ts:14`	One to three paragraphs of prose; well under the model limit.
Texts per Workers AI request	100	`etl/embeddings.ts:14`	Confirmed live; the batch is metered at its longest text.
Batches per embed run	5	`etl/embed.ts:21`	500 passages per invocation, then a continuation job.
Threads per transform run	10	`etl/transform.ts:17`	Tokenizing is the only CPU-heavy step; small runs survive the free plan.
Semantic thresholds	0.80 / 0.70	`api/search.ts:22–23`	High confidence at or above 0.80; weak between; nothing below 0.70.
Neurons per million tokens	1,841	`etl/embeddings.ts:16`	Workers AI's unit of metering; 10,000 neurons per day are free.
Price per million tokens	$0.02	`etl/embeddings.ts:17`	500 passages of p-limit cost $0.00106.
Queue consumer batch size	1	`wrangler.jsonc:28`	The CPU limit applies per invocation, so one job per invocation.
Cron	*/5 min	`wrangler.jsonc:23`	Reconcile lost messages, queue syncs, measure storage.
Job lease	300 s	`queue/jobs.ts:87`	Another consumer takes over only after the lease expires.

3. Technology stack

Technology	Version	Role in ContextTrace	Read
Cloudflare Workers (Free plan)	wrangler 4.149, compatibility date 2026-10-06	The API, the queue consumer, the cron and the static SPA, in one module.	`wrangler.jsonc`, `worker/src/index.ts`
Cloudflare Queues	included in Workers Free since 2026-02	One queue, `contexttrace-jobs`; messages carry job ids and a generation.	`worker/src/queue/jobs.ts`, `consumer.ts`
Cloudflare Cron Triggers		Every five minutes: reconcile, syncs, rescans, retention, storage gauge.	`worker/src/cron/*.ts`
Cloudflare Workers AI	model `@cf/baai/bge-small-en-v1.5`	Embeds passages and queries, 384 dimensions, cls pooling, over the REST API.	`worker/src/etl/embeddings.ts`
Cloudflare Rate Limiting binding		Per-address limits in front of search, registration and reads.	`worker/src/api/ratelimit.ts`
Cloudflare Turnstile	optional	Bot check on registration, verified server side.	`worker/src/api/turnstile.ts`
Workers Static Assets		Serves the built SPA; `/api/*` runs the Worker first.	`wrangler.jsonc:10-14`
Supabase Postgres + pgvector	supabase-js 2.117.3	All records, passages, `vector(384)` embeddings with an HNSW cosine index, budgets, and the SQL functions the Worker calls over PostgREST RPC. Row Level Security on; only the service role may read or write.	`migrations/*.sql`
Postgres full-text search		Keyword baseline: GIN index on `to_tsvector`, `websearch_to_tsquery`, `ts_rank_cd`.	`migrations/20261013170005_keyword_search.sql`
Hugging Face tokenizers (JS)	@huggingface/tokenizers 0.2.0	Counts model tokens with the model's own vendored WordPiece tokenizer.	`worker/src/etl/tokenizer.ts`
GitHub REST API		The only data source: list endpoints for issues, PRs and comments, 100 per page.	`worker/src/etl/github-rest.ts`, `extract.ts`

React + Vite + Cloudflare Vite plugin	React 19, Vite 8, plugin 1.63	The SPA and the local dev server that runs the Worker, the local queue and the consumer in one process.	`vite.config.ts`, `web/src`
Vitest + Cloudflare Vitest plugin	Vitest 5, plugin 1.4	Tests run inside the Workers runtime (workerd) against the live Supabase project; GitHub, Workers AI and Turnstile are stubbed.	`vitest.config.ts`, `tests/integration`
TypeScript, oxlint	TS 6.0, oxlint 1.81	Types generated from Wrangler for the environment; lint.	`worker-configuration.d.ts`
GitHub Pages		A second copy of the SPA; it calls the Worker across origins with CORS and a CSP meta tag.	`.github/workflows/pages.yml`, `api/cors.ts`
Python prototype	not deployed	The readable specification that the Worker's normalize, chunk and passages modules port. It counted tokens with tiktoken; the Worker counts with the model's tokenizer.	`pipeline/*.py`

> **Decisions behind the stack.** `docs/DECISIONS.md` records ten architecture decision records. ADR-0001 chose one Supabase Postgres project with pgvector over D1 plus a separate vector store, so passages and vectors share one consistency boundary. ADR-0002 chose Workers Free with Queues and one job per invocation. ADR-0003 chose bge-small-en-v1.5 over the REST API with cls pooling. ADR-0006 set the 0.80 and 0.70 thresholds and the keyword baseline. ADR-0008 set every budget at 80% of the free allowance. Read the file once before the tracks.

4. Track A: Cloudflare Workers

A Worker is a JavaScript module that Cloudflare runs at the edge. ContextTrace has one Worker. It answers HTTP requests, consumes a queue and runs on a schedule. This track reads the configuration first, then the module, then each binding.

A.1 The Worker configuration: wrangler.jsonc

`wrangler.jsonc`, lines 5 to 45 (whole file, comments shortened)

```
{
  "name": "contexttrace",
  "main": "worker/src/index.ts",
  "compatibility_date": "2026-10-06",
  "assets": {
    "not_found_handling": "single-page-application",
    "run_worker_first": ["/api/*"]
  },
  "vars": {
    "SUPABASE_URL": "https://yqnlcfpvtnnvrhdswoub.supabase.co",
    "ALLOWED_ORIGINS": "https://pudasainimiraj.github.io"
  },
  "triggers": { "crons": ["*/5 * * * *"] },
  "queues": {
    "producers": [{ "binding": "JOBS_QUEUE", "queue": "contexttrace-jobs" }],
    "consumers": [{ "queue": "contexttrace-jobs", "max_batch_size": 1, "max_retries": 3 }]
  },
  "ratelimits": [
    { "name": "SEARCH_LIMITER",   "namespace_id": "1001", "simple": { "limit": 30,  "period": 60 } },
    { "name": "REGISTER_LIMITER", "namespace_id": "1002", "simple": { "limit": 5,   "period": 60 } },
    { "name": "READ_LIMITER",     "namespace_id": "1003", "simple": { "limit": 120, "period": 60 } }
  ],
  "observability": { "enabled": true },
  "upload_source_maps": true
}
```

What to learn from each key:

- `main` names the module whose default export holds the handlers.
- `assets` makes Wrangler upload the Vite build as static files. `run_worker_first` is the rule that keeps `/api/*` away from the SPA fallback; everything else that is not a file falls back to `index.html`.
- `vars` are public and committed. Secrets are never here; they are set with `wrangler secret put` and read from `env` at run time (see A.3).
- `queues.producers` gives the Worker a binding called `JOBS_QUEUE` with a `send()` method. `queues.consumers` makes the same Worker the consumer of that queue; `max_batch_size: 1` delivers one message per invocation.
- `ratelimits` declares three counters as bindings, each with a limit per period.
- `triggers.crons` schedules the `scheduled()` handler.

A.2 One module, three entry points: fetch, scheduled, queue

`worker/src/index.ts`, lines 27 to 94

```
type App = { db: SupabaseClient; budget: Budget; queue: JobQueue; meter: EgressMeter; env: Env };

function createApp(env: Env): App {
  const meter: EgressMeter = { bytes: 0 };
  const db = createDb(env, meter);
  const budget = new Budget(db, env.BUDGET_SCOPE || GLOBAL_SCOPE);
  return { db, budget, queue: meteredQueue(env.JOBS_QUEUE, budget), meter, env };
}

export default {
```

```
  async fetch(request, env, ctx): Promise<Response> {
    const origin = allowedOrigin(request, env.ALLOWED_ORIGINS);
    if (request.method === "OPTIONS" && origin !== null) return preflightResponse(origin);
    const app = createApp(env);
    try {
      return withCors(await route(request, app), origin);
    } catch (err) {
      console.error("unhandled request error", err);
      return withCors(error(500, "internal_error", "Something went wrong on our side; ..."), origin);
    } finally {
      settleEgress(app, ctx);
    }
  },

  async scheduled(_controller, env, ctx): Promise<void> {
    const app = createApp(env);
    try {
      const reconciled = await reconcileJobs(app.db, app.queue);
      const syncs = await scheduleSyncs(app.db, app.queue, { budget: app.budget });
      const rescans = await scheduleRescans(app.db, app.queue, { budget: app.budget });
      const catchUps = await scheduleCatchUps(app.db, app.queue, { budget: app.budget });
      const maintenance = await maintain(app.db, app.budget);
      console.log("cron", { reconciled, syncs, rescans, catchUps, maintenance });
    } finally {
      settleEgress(app, ctx);
    }
  },

  async queue(batch, env, ctx): Promise<void> {
    const app = createApp(env);
    const jobCtx = { db: app.db, queue: app.queue, budget: app.budget,
      token: env.GITHUB_TOKEN || undefined, embeddings: createEmbeddingClient(env) };
    const owner = `consumer:${crypto.randomUUID()}`;
    try {
      for (const message of batch.messages) {
        await consumeJobMessage(jobCtx, message, owner);
      }
    } finally {
      settleEgress(app, ctx);
    }
  },
} satisfies ExportedHandler<Env, JobMessage>;
```

Points to notice:

- The default export is an object with three handlers. Cloudflare calls `fetch` for HTTP, `scheduled` for the cron and `queue` for queue deliveries. `satisfies ExportedHandler<Env, JobMessage>` type-checks all three against the runtime's types, with `JobMessage` as the queue message type.
- Every handler receives `env`, the object that holds the bindings and secrets, and `ctx`, the execution context. Nothing is kept between invocations; `createApp` builds the clients fresh every time.
- `ctx.waitUntil(promise)` in `settleEgress` (line 39) lets work finish after the response has been sent. Here it settles the bytes Supabase sent against the egress budget.
- The `fetch` handler never lets an exception reach the client. The error goes to the log; the client gets a generic 500 with the CORS headers it needs.
- The `queue` handler creates one owner id per invocation. The consumer uses it to lease jobs (A.4).

The router at lines 96 to 145 is a plain chain of `if` statements over `method` and `pathname` with regular expressions for the ids. There is no framework. A rate limit or an admin check runs before the handler it protects, and a `null` from the check means "allowed".

A.3 The typed environment: bindings, vars and secrets

`worker-configuration.d.ts`, lines 4 to 28 (generated by `wrangler types --env-file .env.example`)

```
interface __BaseEnv_Env {
  JOBS_QUEUE: Queue;
  SEARCH_LIMITER: RateLimit;
  REGISTER_LIMITER: RateLimit;
  READ_LIMITER: RateLimit;
  ALLOWED_ORIGINS: "https://pudasainimiraj.github.io";
  GITHUB_TOKEN: string;
  SUPABASE_URL: string;
  SUPABASE_SECRET_KEY: string;
  SUPABASE_DB_URL: string;
  CLOUDFLARE_ACCOUNT_ID: string;
  WORKERS_AI_API_TOKEN: string;
  ADMIN_TOKEN: string;
  BUDGET_SCOPE: string;
  TURNSTILE_SECRET_KEY: string;
  VITE_TURNSTILE_SITE_KEY: string;
  VITE_API_BASE_URL: string;
}
interface Env extends __BaseEnv_Env {}
```

Wrangler reads `wrangler.jsonc` and the names in `.env.example` and writes this file. The bindings (`Queue`, `RateLimit`) get their runtime types; the vars and secrets become strings. The `Env` type is what every handler and every `createX(env)` helper takes. Compare `createEmbeddingClient(env: Pick<Env, "CLOUDFLARE_ACCOUNT_ID" | "WORKERS_AI_API_TOKEN">)` in `etl/embeddings.ts:124`: the helper names only the two keys it needs, so a test can pass a small object.

Kind	Local development	Deployed Worker	Examples
Var (public)	`wrangler.jsonc` `vars`	same file, uploaded on deploy	`SUPABASE_URL`, `ALLOWED_ORIGINS`
Secret	`.env` at the root, gitignored	`npx wrangler secret put NAME`, once	`SUPABASE_SECRET_KEY`, `WORKERS_AI_API_TOKEN`, `CLOUDFLARE_ACCOUNT_ID`, `GITHUB_TOKEN`, `ADMIN_TOKEN`
Binding	local emulation by the Vite plugin (Miniflare)	declared in `wrangler.jsonc`	`JOBS_QUEUE`, `SEARCH_LIMITER`

> **Workers AI token.** The token for the REST API is a Workers AI token, not the general `CLOUDFLARE_API_TOKEN`. The account id is also needed because the REST URL embeds it. Both are secrets; when either is missing, `createEmbeddingClient` returns `undefined` and every embed job fails at once with `ai_not_configured` (`etl/embed.ts:69–74`).

A.4 Cloudflare Queues: producer and consumer

The message

worker/src/queue/jobs.ts, lines 7 to 28

```
export type JobMessage = { jobId: number; repoId: number; kind: string; generation: number };

/** Longest delay a message is sent with: half the free plan's 24-hour retention. */
export const MAX_MESSAGE_DELAY_SECONDS = 12 * 3600;

/** The part of a Cloudflare Queue producer binding that enqueueJob uses. */
export type JobQueue = { send(message: JobMessage, options?: { delaySeconds?: number }): Promise<unknown>
};

/** A queue whose sends reserve queue operations first. */
export function meteredQueue(queue: JobQueue, budget: Budget): JobQueue {
  return {
    async send(message, options) {
      await budget.take("queue_ops", QUEUE_OPS_PER_MESSAGE);
      return queue.send(message, options);
    },
  };
}
```

A message is 76 bytes. It carries ids and a generation, nothing else. The job row in Postgres holds the state, the attempt count, the lease and the result. This is the central design lesson of the queue: **the queue delivers, the database decides**. The `JobQueue` type names only the `send` method of the binding, so tests pass a `vi.fn()` and production passes `env.JOBS_QUEUE` wrapped by `meteredQueue`, which reserves three queue operations (write, read, delete) per message against the free plan's 10,000 per day.

Producer: enqueueJob

worker/src/queue/jobs.ts, lines 43 to 62

```
export async function enqueueJob(db, queue, jobId, options = {}): Promise<JobRecord | null> {
  const { data, error } = await db.rpc("enqueue_job", { p_job_id: jobId }).maybeSingle();
  if (error) throw dbError("enqueue_job", error);
  if (!data) return null;
  const job = data as JobRecord;
  const message: JobMessage = { jobId: job.id, repoId: job.repository_id, kind: job.kind, generation:
job.generation };
  const delaySeconds = Math.min(options.delaySeconds ?? 0, MAX_MESSAGE_DELAY_SECONDS);
  try {
    await queue.send(message, delaySeconds ? { delaySeconds } : undefined);
  } catch (err) {
    if (!isBudgetExhausted(err)) throw err;
    console.warn(`job ${job.id} stays queued without a message: ${err.message}`);
  }
  return job;
}
```

The SQL function `enqueue_job` bumps the row's generation and returns it. The message carries that generation. A later enqueue of the same job makes every earlier message stale, and the consumer can tell. `delaySeconds` is the queue's own delayed delivery: backfills wait 10 s, retries wait their backoff, a budget pause waits until the window resets. If the queue budget refuses the send, the row stays `queued` and the cron's reconcile sweep sends it later.

Consumer: consumeJobMessage

worker/src/queue/consumer.ts, lines 106 to 163

```
export async function consumeJobMessage(ctx: ConsumerContext, message: Message<unknown>, owner: string):
Promise<void> {
  if (!isJobMessage(message.body)) {
    console.warn("ignoring malformed job message", message.id);
    message.ack();
```

```
    return;
  }

  const claim = await claimJob(ctx.db, message.body, owner);
  if (claim.outcome === "not_due") {
    const waitMs = Date.parse(claim.job.next_attempt_at ?? "") - Date.now();
    message.retry({ delaySeconds: Math.max(1, Math.ceil(waitMs / 1000)) });
    return;
  }
  if (claim.outcome !== "claimed") {
    message.ack();
    return;
  }

  const job = claim.job;
  const run = ctx.run ?? runJob;
  try {
    const startedAt = Date.now();
    const outcome = await run(job, ctx);
    const result = { ...outcome.result, durationMs: Date.now() - startedAt };
    await finishJob(ctx.db, job.id, owner, { state: "succeeded", result });
    if (!outcome.done) await ensureQueuedJob(ctx.db, ctx.queue, job.repository_id, job.kind,
      { delaySeconds: continuationDelaySeconds(job.kind) });
    await queueFollowUps(ctx, job, outcome.followUps ?? [], outcome.changed ?? true);
  } catch (err) {
    const failure = classifyError(err);
    const paused = failure.code === "budget_exhausted";
    if (paused || (failure.transient && job.attempt < MAX_ATTEMPTS)) {
      const delaySeconds = paused ? Math.max(60, failure.retryAfterSeconds ?? 60)
                                  : backoffSeconds(job.attempt, failure.retryAfterSeconds);
      const parked = await finishJob(ctx.db, job.id, owner, {
        state: "retry_pending", error_code: failure.code, error_message: failure.message,
        next_attempt_at: new Date(Date.now() + delaySeconds * 1000).toISOString(),
      });
      if (parked) await enqueueJob(ctx.db, ctx.queue, job.id, { delaySeconds });
    } else {
      await finishJob(ctx.db, job.id, owner, {
        state: "failed", error_code: failure.code, error_message: failure.message,
      });
    }
  }
  message.ack();
}
```

The two queue primitives are `message.ack()` and `message.retry({ delaySeconds })`. The consumer acks in every case except one: a job that is parked and not yet due is retried with a delay, so the queue redelivers it when the backoff ends. Everything else, including a failure, is acked, because the job row already records what happened and a new message was sent if needed. The consumer never relies on the queue's own `max_retries`; it is a safety net.

The claim (`jobs.ts:83–106`) is an atomic lease in SQL. The same owner seeing the same message again is claimed again, so a redelivery is safe. Another owner gets the job only after the 300 s lease expires. `finishJob` (`jobs.ts:118–134`) updates the row only when the owner still holds the lease, so a late finish by a worker that lost its lease cannot overwrite the takeover's result.

A run returns `{ done, result, followUps, changed }`. `done: false` asks for a continuation job of the same kind. `changed: false` suppresses the next job kind: a sync that found nothing does not transform, and a transform that touched nothing does not embed. On the free plan every message counts.

> **Why one job per invocation.** The free plan's CPU limit applies per invocation. With `max_batch_size: 1` every message is its own invocation, so a long transform cannot starve the jobs behind it. The `for` loop over `batch.messages` in `index.ts:87` stays, because the local dev queue batches messages anyway.

A.5 Cron Triggers

The `scheduled` handler (A.2) runs every five minutes. In order: `reconcileJobs` resends messages for jobs whose lease expired or whose message was lost (`cron/reconcile.ts`); `scheduleSyncs` queues an hourly `fetch_<stream>` per complete stream and `scheduleRescans` a weekly rescan (`cron/sync.ts`); `scheduleCatchUps` queues transforms and embeds that a hand-off missed; `maintain` purges history, removes repositories unreachable for 30 days and measures storage into the gauge (`cron/maintenance.ts`). The schedulers defer when less than 20% of the day's queue budget remains, which is how the service avoids spending its last messages on background work.

A.6 Living inside the free plan CPU limit

The free plan documents 10 ms of CPU per invocation. ADR-0002 records what was measured: transforms of 44 to 749 ms CPU ran fine on small repositories, but on tensorflow/tensorflow runs of 1.4 to 2 s were terminated with `exceededCpu`, and once the tolerance was spent even 116 ms runs were cut. The limit is a tolerance, not a hard number. Three techniques keep ContextTrace inside it.

worker/src/etl/transform.ts, lines 17 to 26

```
export const DOCUMENTS_PER_RUN = 10;

/** Threads for a run at the given attempt: the full run first, then half as many on every retry. */
export function documentsPerRun(attempt: number): number {
  return Math.max(1, Math.floor(DOCUMENTS_PER_RUN / 2 ** Math.max(0, attempt - 1)));
}
```

1. **Small runs, chained.** A transform run handles 10 threads and returns `done: false` while dirty threads remain; the consumer queues a continuation. A run the platform killed comes back on the next attempt with half the threads, down to one, so one giant thread cannot block the chain.
2. **Tokenize once.** The splitter in `etl/chunk.ts` counts every part once and computes the count of a join from the parts' counts minus a per-separator overhead (C.4). That cut CPU four to six times.
3. **Cap the input.** A unit body above 20,000 characters is cut before tokenizing (`etl/passages.ts:132`). The discussion comes first in a body; the tail is usually a pasted log.

> **Measuring CPU.** Inside a Worker `Date.now()` only advances across I/O, so every `durationMs` in a job result is wall time, not CPU. True CPU time is visible only on a deployed Worker: Workers Observability or `wrangler tail` reports `cpuTime` per invocation. `docs/benchmarks/ct-24-transform-embed.md` and `docs/milestone-7-acceptance.md` show both.

A.7 Rate limiting, static assets, CORS, Turnstile

- **Rate limiting binding.** `api/ratelimit.ts` calls `limiter.limit({ key })` with the client address from `cf-connecting-ip` and answers 429 when the binding says no. The counters are per Cloudflare location and eventually consistent, so they blunt bursts while the budgets stay the hard caps (ADR-0009). Local development has no client address and is not limited.
- **Static assets.** The SPA build is uploaded with the Worker. Unknown paths get `index.html`; `/api/*` reaches the Worker first. `web/public/_headers` sets a strict Content Security Policy for the assets.
- **CORS.** The GitHub Pages copy of the SPA calls the Worker across origins. `api/cors.ts` answers preflights for the origins in `ALLOWED_ORIGINS` and adds the headers to every response for that origin, the 500 included.
- **Turnstile.** When `TURNSTILE_SECRET_KEY` is set, `api/turnstile.ts` verifies the token the browser got from the widget before any budget is spent (`index.ts:186–198`).

A.8 Developing and testing a Worker locally

vite.config.ts, lines 29 to 33, and **vitest.config.ts**

```
// vite.config.ts: the Worker with the SPA as its static assets
return {
  root: "web",
  build: { outDir: "../dist", emptyOutDir: true },
  plugins: [react(), cloudflare({ configPath: fileURLToPath(new URL("./wrangler.jsonc", import.meta.url))
})],
};

// vitest.config.ts: tests run inside the Workers runtime
export default defineConfig({
  plugins: [cloudflareTest({ wrangler: { configPath: "./wrangler.jsonc" } })],
  test: { include: ["tests/**/*.test.ts"], fsModuleCache: false, testTimeout: 15_000 },
});
```

The Cloudflare Vite plugin runs the Worker in workerd, the same runtime as production, together with a local queue and its consumer, in one `npm run dev` process. The Vitest plugin does the same for tests: a test file imports `env` from `cloudflare:workers` and gets the bindings and the values of `.env`.

tests/integration/embeddings.test.ts, lines 1 and 191 to 195

```
import { env } from "cloudflare:workers";
…
describe.skipIf(!env.CLOUDFLARE_ACCOUNT_ID || !env.WORKERS_AI_API_TOKEN)("Workers AI live", () => {
  it("embeds three texts with cls pooling into 384 finite dimensions", async () => {
    const client = createEmbeddingClient(env)!;
```

Two testing patterns run through the code base. First, **injectable I/O**: the embedding client takes a `fetch` option, the GitHub client too, so a unit test passes `vi.fn()` and asserts on the request. Second, **live tests gated on credentials**: a `describe.skipIf` skips the real Workers AI call when the secrets are absent. The database tests are not stubbed; they run against the live Supabase project with repository ids above 1e12 and delete their rows in `afterAll`.

Command	What it does
`nvm use 22 && npm run dev`	Vite dev server with the Worker, the local queue and the consumer on port 5173.
`npm run test`	Vitest inside workerd; most files need `SUPABASE_SECRET_KEY` in `.env`.
`npm run typecheck`, `npm run lint`	`tsc -b` over the three tsconfig projects; oxlint.
`npm run cf-typegen`	Regenerates `worker-configuration.d.ts` after a binding or secret name changes.
`npm run eval`	The retrieval evaluation against the public API, no secrets needed.

A.9 Deploying

```
npx vite build && npx wrangler deploy           # or: npm run deploy
npx wrangler secret put SUPABASE_SECRET_KEY     # once per secret
npx wrangler secret put WORKERS_AI_API_TOKEN
npx wrangler secret put CLOUDFLARE_ACCOUNT_ID
npx wrangler secret put GITHUB_TOKEN
npx wrangler secret put ADMIN_TOKEN
npx wrangler tail                                # live logs with cpuTime per invocation
```

The Vite build writes the Worker bundle and a resolved `wrangler.json` into `dist/contexttrace`, and the SPA into `dist/client`. `wrangler deploy` uploads both, the queue bindings, the cron and the rate limiters. `GET /api/status` on the deployed Worker reports which protections are on and how much of every budget is used.

5. Track B: Embeddings with Workers AI

An embedding is a list of numbers that places a text in a space where nearby points mean similar things. ContextTrace embeds every passage once, stores the vector, and at search time embeds the question with the same model and asks the database for the nearest stored vectors. This track reads the client that calls the model, the job that drives it, and the tokenizer that keeps the inputs inside the model's limit.

B.1 What the model does

worker/src/etl/embeddings.ts, lines 8 to 18

```
export const EMBEDDING_MODEL = "@cf/baai/bge-small-en-v1.5" as const;
export const EMBEDDING_POOLING = "cls" as const;
export const EMBEDDING_DIMS = 384;
/** The model's sequence length; passages.token_count already includes [CLS] and [SEP]. */
export const EMBEDDING_MAX_TOKENS = 512;
/** Texts per request. The documented ceiling is unofficial; the live CT-23 test confirms it. */
export const EMBEDDING_BATCH_SIZE = 100;
/** Workers AI pricing for this model: $0.020 per million input tokens. */
export const NEURONS_PER_MILLION_TOKENS = 1841;
export const USD_PER_MILLION_TOKENS = 0.02;
export const WORKERS_AI_API_ROOT = "https://api.cloudflare.com/client/v4";
```

`bge-small-en-v1.5` is a BERT-style encoder from the Beijing Academy of Artificial Intelligence. It reads up to 512 WordPiece tokens and produces a 384-dimensional vector. Two vectors are compared with cosine similarity: 1 means the same direction, 0 means unrelated. Similar texts get similar vectors, so a question and the comment that answers it land close together even when they share no words.

Pooling is how the model's per-token outputs become one vector. `cls` takes the output at the `[CLS]` token; `mean` averages all tokens. The model was trained for `cls`, Workers AI defaults to `mean`, and vectors made with different poolings are not comparable. ADR-0003 therefore pins `cls` in the request, checks the server's answer, and pins it again in the database (C.7). The query at search time is embedded the same way (B.8); no model ever runs in the browser.

B.2 The REST client

worker/src/etl/embeddings.ts, lines 52 to 85

```
export class WorkersAiEmbeddingClient implements EmbeddingClient {
  private readonly url: string;
  private readonly apiToken: string;
  private readonly fetchImpl: typeof globalThis.fetch;

  constructor(options: WorkersAiOptions) {
    const root = options.apiRoot ?? WORKERS_AI_API_ROOT;
    this.url = `${root}/accounts/${encodeURIComponent(options.accountId)}/ai/run/${EMBEDDING_MODEL}`;
    this.apiToken = options.apiToken;
    this.fetchImpl = options.fetch ?? ((input, init) => globalThis.fetch(input, init));
  }

  async embed(texts: string[]): Promise<EmbeddingResult> {
    if (texts.length === 0) return { model: EMBEDDING_MODEL, pooling: EMBEDDING_POOLING, vectors: [],
durationMs: 0 };
    if (texts.length > EMBEDDING_BATCH_SIZE) {
      throw new JobError("ai_bad_request", `A batch of ${texts.length} texts exceeds
${EMBEDDING_BATCH_SIZE}`, { transient: false });
    }

    const started = Date.now();
    let res: Response;
    try {
      res = await this.fetchImpl(this.url, {
        method: "POST",
```

```
        headers: {
          Authorization: `Bearer ${this.apiToken}`,
          "content-type": "application/json",
          "User-Agent": "contexttrace-worker",
        },
        body: JSON.stringify({ text: texts, pooling: EMBEDDING_POOLING }),
      });
    } catch (err) {
      throw new JobError("network", err instanceof Error ? err.message : String(err),
        { transient: true });
    }
    const durationMs = Date.now() - started;
    …
```

The whole call is one HTTPS request:

```
POST https://api.cloudflare.com/client/v4/accounts/{ACCOUNT_ID}/ai/run/@cf/baai/bge-small-en-v1.5
Authorization: Bearer {WORKERS_AI_API_TOKEN}
content-type: application/json

{ "text": ["first passage", "second passage", "..."], "pooling": "cls" }
```

The body is literally `{ text, pooling: "cls" }`. There is no model or pooling parameter anywhere in the class, so nothing can mix a second model into the table by accident. The `EmbeddingClient` interface (line 33) has one method, `embed(texts)`; the rest of the code depends on the interface, which is why a fake client can replace it in tests (B.9).

> **REST API or AI binding?** A Worker can also declare an `ai` binding in `wrangler.jsonc` and call `env.AI.run("@cf/baai/bge-small-en-v1.5", { text, pooling: "cls" })`. ContextTrace does not; ADR-0003 explains that the binding runs remotely anyway, and the REST client gives an injectable `fetch`, the raw envelope with `usage`, and the same code path for a test and for production. If you add the binding in your own project, keep the pooling explicit.

B.3 Reading the response

`worker/src/etl/embeddings.ts`, lines 44 to 50 and 104 to 119

```
type Usage = { total_tokens?: unknown; prompt_tokens?: unknown };
type Envelope = {
  success?: boolean;
  errors?: Array<{ message?: string }>;
  usage?: Usage;
  result?: { shape?: unknown; data?: unknown; pooling?: unknown; usage?: Usage };
};
…
    const result = body.result;
    const data = result?.data;
    if (!Array.isArray(data) || data.length !== texts.length) {
      throw new JobError("ai_invalid_output", `Expected ${texts.length} vectors from Workers AI`,
        { transient: false });
    }
    const shape = result?.shape;
    if (Array.isArray(shape) && (shape[0] !== texts.length || shape[1] !== EMBEDDING_DIMS)) {
      throw new JobError("ai_invalid_output", `Unexpected embedding shape ${JSON.stringify(shape)}`,
        { transient: false });
    }
    if (result?.pooling !== undefined && result.pooling !== EMBEDDING_POOLING) {
      throw new JobError("ai_pooling_mismatch", `Workers AI answered with ${String(result.pooling)}
pooling`,
        { transient: false });
    }
    const usage = result?.usage ?? body.usage;
    const metered = usage?.total_tokens ?? usage?.prompt_tokens;
    const tokens = typeof metered === "number" && Number.isFinite(metered) ? metered : undefined;
```

```
    return { model: EMBEDDING_MODEL, pooling: EMBEDDING_POOLING, vectors: data as number[][], durationMs,
tokens };
```

Workers AI wraps every answer in the Cloudflare API envelope: `success`, `errors`, `messages`, `result`. For this model, `result.shape` is `[texts, 384]`, `result.data` is the array of vectors aligned with the input, `result.pooling` echoes the pooling, and `usage.total_tokens` is the metered input. The client checks every one of those before it trusts the vectors: the count must match, the shape must match, the pooling must be `cls`. Each check has a test in `tests/integration/embeddings.test.ts` (lines 130 to 141).

B.4 Error mapping

Every failure becomes a `JobError` with a code and a `transient` flag. The consumer (A.4) retries transient errors with backoff and fails permanent ones at once. One case is special: the free plan's daily allocation comes back as a 429 with a message, and the client turns it into a budget pause until 00:00 UTC rather than a retry.

worker/src/etl/embeddings.ts, lines 87 to 102

```
    const body = (await readJson(res)) as Envelope | undefined;
    const message = body?.errors?.[0]?.message ?? `Workers AI responded ${res.status}`;
    if (res.status === 429) {
      if (/daily free allocation/i.test(message)) {
        throw new JobError("budget_exhausted", `Workers AI: ${message}`,
          { transient: true, retryAfterSeconds: secondsUntilUtcMidnight() });
      }
      throw new JobError("ai_rate_limited", message,
        { transient: true, retryAfterSeconds: retryAfterSeconds(res) });
    }
    if (res.status === 401 || res.status === 403) throw new JobError("ai_auth", message, { transient:
false });
    if (res.status >= 500) throw new JobError("ai_upstream", message, { transient: true });
    if (!res.ok || body === undefined || body.success === false) {
      throw new JobError("ai_bad_request", message, { transient: false });
    }
```

Condition	Code	Transient	What happens to the job
429 with "daily free allocation" in the message	`budget_exhausted`	yes	Parked until 00:00 UTC; the attempt is given back.
Other 429	`ai_rate_limited`	yes	Parked for `Retry-After` seconds (default 60).
401 or 403	`ai_auth`	no	Fails on attempt 1; fix the token or the account id.
5xx	`ai_upstream`	yes	Backoff from 30 s doubling to 15 min, failed after 5 attempts.
fetch threw	`network`	yes	Same backoff.
400 or `success: false`	`ai_bad_request`	no	Fails; the API message is kept.
Missing or misaligned data, wrong shape	`ai_invalid_output`	no	Fails; nothing stored.
Pooling other than cls	`ai_pooling_mismatch`	no	Fails; nothing stored.
One bad vector inside a good batch	per passage		Only that passage is counted invalid and stays pending; the batch's other vectors are stored (B.7).
No credentials	`ai_not_configured`	no	Fails on attempt 1.

B.5 Metering, neurons and cost

Workers AI meters in **neurons**. For this model, 1,841 neurons buy one million input tokens, and the price is $0.02 per million. The free plan gives 10,000 neurons per day, which resets at 00:00 UTC. ContextTrace keeps its own ledger so that it can refuse work before the provider does.

worker/src/etl/embeddings.ts, lines 142 to 148, and **worker/src/queue/budget.ts**, lines 30 to 33

```
export function neuronsFor(tokens: number): number { return (tokens * NEURONS_PER_MILLION_TOKENS) / 1e6;
}
export function usdFor(tokens: number): number { return (tokens * USD_PER_MILLION_TOKENS) / 1e6; }

/** Workers AI meters 1,841 neurons per million input tokens: 1.841 milli-neurons per token. */
export function milliNeuronsFor(tokens: number): number {
  return Math.ceil((tokens * NEURONS_PER_MILLION_TOKENS) / 1000);
}
```

The ledger counts in milli-neurons so that it can use integers. The embed job reserves before the call and settles after it:

worker/src/etl/embed.ts, lines 98 to 116

```
for (const batch of chunk(ordered, batchSize)) {
  // Workers AI meters every text at the batch's longest, so that is what is reserved;
  // the reservation is settled to the metered count once the response reports it.
  const estimatedTokens = batch.length * Math.max(...batch.map((p) => p.token_count));
  if (ctx.budget) await ctx.budget.take("ai_neurons_milli", milliNeuronsFor(estimatedTokens));
  let result;
  try {
    result = await client.embed(batch.map((p) => p.text));
  } catch (err) {
    // A call that failed was not metered by Workers AI: the reservation goes back.
    if (ctx.budget) await ctx.budget.settle("ai_neurons_milli", -milliNeuronsFor(estimatedTokens));
    throw err;
  }
  …
  totals.billedTokens += result.tokens ?? estimatedTokens;
  if (ctx.budget && result.tokens !== undefined)
    await ctx.budget.settle("ai_neurons_milli", milliNeuronsFor(result.tokens) -
milliNeuronsFor(estimatedTokens));
```

The longest-text rule. The benchmark in `docs/benchmarks/ct-24-transform-embed.md` sent three texts of 11, 11 and 8 tokens and was billed 33: every text in a batch is metered at the batch's longest text. The job therefore sorts a run's passages by `token_count` before batching (`embed.ts:95`). Even so, the first p-limit run billed 52,788 tokens for 41,515 exact tokens, a 27% padding, because the sorted 500 still spanned 10 to 256 tokens across five batches of 100. Smaller batches of more uniform length would cut it at the price of more requests.

`reserve_budget` and `settle_budget` are SQL functions (`migrations/20261014164438_usage_budgets.sql`), atomic and window-aware, with every limit at 80% of the free allowance. A refused reservation throws `budget_exhausted` with the reset time, and the consumer parks the job without using an attempt. Every run with at least one model call also writes a row to `embedding_usage` with exact tokens, billed tokens and the neuron estimate, so the ledger can be compared with the Cloudflare dashboard at any time.

p-limit (CT-24, 2026-10-13)	Value
Passages embedded	500
Model calls (batches)	6
Exact tokens	41,559
Billed tokens	52,848
Neurons	97.29 of 10,000 per day
Cost	$0.00106
Storage with the HNSW index	2,176 kB, about 4.4 kB per row

> **Open item.** NEXT.md records that on 2026-10-16 Workers AI refused embeddings at its daily allocation while the ledger showed 1,683 of 8,000 neurons. Either the rate of 1,841 neurons per million tokens or the reset window differs from Cloudflare's accounting. Until it is calibrated, the provider's own 429 is what parks the embed jobs.

B.6 Validating a vector

worker/src/etl/embeddings.ts, lines 131 to 140

```
export type VectorProblem = "not_array" | "wrong_dims" | "not_finite" | "all_zero";

export function validateVector(value: unknown, dims = EMBEDDING_DIMS): VectorProblem | null {
  if (!Array.isArray(value)) return "not_array";
  if (value.length !== dims) return "wrong_dims";
  if (!value.every((x) => typeof x === "number" && Number.isFinite(x))) return "not_finite";
  if (value.every((x) => x === 0)) return "all_zero";
  return null;
}
```

A vector that is not 384 finite numbers, or is all zeros, is never stored. The check runs per passage in the embed job and once more for the query vector at search time. A zero vector would have no direction and would break cosine similarity; a NaN would poison the index.

B.7 The embed job

worker/src/etl/embed.ts, lines 63 to 96 (setup) and 118 to 147 (store)

```
export async function embedPassages(ctx: EmbedContext, repositoryId: number, jobId: number,
  options: EmbedOptions = {}): Promise<EmbedResult> {
  const client = ctx.embeddings;
  if (!client) throw new JobError("ai_not_configured", "Workers AI credentials are missing (...)",
    { transient: false });
  const batchSize = options.batchSize ?? EMBEDDING_BATCH_SIZE;
  const budget = batchSize * (options.maxBatches ?? MAX_BATCHES_PER_RUN);

  const rows = await pendingPassages(ctx.db, repositoryId, budget + 1);
  const hasMore = rows.length > budget;
  const { eligible, oversized, empty } = partitionPassages(rows.slice(0, budget));
  …
  const ordered = [...eligible].sort((a, b) => a.token_count - b.token_count || (a.passage_id <
b.passage_id ? -1 : 1));
  let firstBatchStored = false;
  try {
    for (const batch of chunk(ordered, batchSize)) {
      …  // reserve, call, settle (B.5)
      const drafts: PassageEmbeddingDraft[] = [];
      batch.forEach((p, i) => {
        const problem = validateVector(result.vectors[i]);
        if (problem) {
          totals.invalid += 1;
          console.warn(`passage ${p.passage_id}: invalid embedding (${problem})`);
          return;
        }
        drafts.push({
          passage_id: p.passage_id, repository_id: repositoryId,
          model: EMBEDDING_MODEL, pooling: EMBEDDING_POOLING, dims: EMBEDDING_DIMS,
          content_hash: p.content_hash, token_count: p.token_count,
          embedding: result.vectors[i], embedded_by_job: jobId,
        });
      });
      if (drafts.length) {
        const counts = await upsertPassageEmbeddings(ctx.db, drafts);
        totals.inserted += counts.inserted;
        totals.updated += counts.updated;
        if (!firstBatchStored) {
```

```
          firstBatchStored = true;
          await refreshRepositoryStatus(ctx.db, repositoryId, TRANSFORM_VERSION);
        }
      }
    }
  } finally { … record embedding_usage … }
  await refreshRepositoryStatus(ctx.db, repositoryId, TRANSFORM_VERSION);
  return { done: !hasMore, ...totals, neuronsEstimated: neuronsFor(totals.billedTokens) };
}
```

Read the job as six steps:

1. **Ask the database what is pending.** `pending_passages` (C.7) returns passages with no vector, or whose `content_hash` changed since they were embedded. The job asks for one row more than its budget to learn whether more remain.
2. **Partition.** Empty texts and texts above 512 tokens are counted and skipped, never sent, never truncated.
3. **Sort by length, batch by 100.** The longest-text rule (B.5).
4. **Reserve, call, settle.** Per batch.
5. **Validate and store.** One invalid vector fails one passage. A failing batch keeps the batches stored before it, because each batch is its own upsert.
6. **Refresh the status.** After the first stored batch the repository is `searchable_partial`; at the end it may become `searchable`. `done` is decided by the run budget, never by what is still pending, so a passage that can never be embedded cannot loop forever.

The consumer registers the job under the kind `embed_passages` (`queue/consumer.ts:42–45`) and queues it after every transform run that changed something (`queue/pipeline.ts:20–23`).

B.8 Embedding the search query

`worker/src/api/search.ts`, lines 44 to 63

```
export async function embedQuery(embeddings: EmbeddingClient | undefined, query: string, budget?: Budget)
  : Promise<{ vector: number[]; durationMs: number }> {
  if (!embeddings) throw new JobError("ai_not_configured", "No embedding client is configured",
    { transient: false });
  if (budget) await budget.take("ai_neurons_milli", QUERY_RESERVE_MILLI_NEURONS);
  let result;
  try {
    result = await embeddings.embed([query]);
  } catch (err) {
    if (budget) await budget.settle("ai_neurons_milli", -QUERY_RESERVE_MILLI_NEURONS);
    throw err;
  }
  if (budget && result.tokens !== undefined)
    await budget.settle("ai_neurons_milli",
      milliNeuronsFor(result.tokens) - QUERY_RESERVE_MILLI_NEURONS);
  const vector = result.vectors[0];
  const problem = validateVector(vector);
  if (problem) throw new JobError("ai_invalid_output", `Query vector is ${problem}`,
    { transient: false });
  return { vector, durationMs: result.durationMs };
}
```

The same client, the same model, the same pooling, the same validation. A query reserves 200 milli-neurons before its real token count is known, then settles. Queries are limited to 500 characters before they reach the model (`search.ts:15`). When the client throws, `semanticSearch` (lines 182 to 197) catches the `JobError`, logs it and returns the error code, so the search falls back to keywords instead of failing (C.10).

B.9 Testing without the model

worker/src/etl/embeddings.ts, lines 150 to 177

```
/** Deterministic test double: the text seeds a unit vector, so equal texts embed equally. */
export class FakeEmbeddingClient implements EmbeddingClient {
  readonly calls: string[][] = [];
  …
  async embed(texts: string[]): Promise<EmbeddingResult> {
    const call = this.calls.length;
    this.calls.push(texts);
    const failure = this.fail?.(texts, call);
    if (failure) throw failure;
    const vectors = texts.map((text, index) => this.vectorFor?.(text, index, call) ?? fakeVector(text));
    return { model: EMBEDDING_MODEL, pooling: EMBEDDING_POOLING, vectors, durationMs: 0 };
  }
}

/** FNV-1a seeds mulberry32; the values are scaled to a unit vector. */
export function fakeVector(text: string, dims = EMBEDDING_DIMS): number[] {
  const next = mulberry32(fnv1a(text));
  const values = Array.from({ length: dims }, () => next() * 2 - 1);
  const norm = Math.hypot(...values) || 1;
  return values.map((x) => x / norm);
}
```

The fake hashes the text with FNV-1a, seeds a small random generator with the hash, draws 384 values and normalizes them to length 1. Equal texts get equal vectors, different texts get different ones, and the vectors are valid pgvector input. That is enough for every integration test of the embed job, the search function and the repository isolation (`tests/integration/search.test.ts`) to run against the real database without a model. The `vectorFor` and `fail` hooks inject a NaN vector or a 429 for one call. The live test (`embeddings.test.ts:191–212`) checks what the fake cannot: identical texts embed identically, an unrelated text scores below 0.9, and the metered tokens sit between the exact sum and the longest-text product.

B.10 Counting tokens the way the model does

worker/src/etl/tokenizer.ts, whole file

```
import { Tokenizer } from "@huggingface/tokenizers";
import tokenizerConfigRaw from "./tokenizer/bge-small-en-v1.5/tokenizer_config.json?raw";
import tokenizerJsonRaw from "./tokenizer/bge-small-en-v1.5/tokenizer.json?raw";

export const TOKENIZER_NAME = "BAAI/bge-small-en-v1.5";
export const MODEL_MAX_TOKENS = 512;

let instance: Tokenizer | undefined;

/** Lazily built singleton; the first call parses the 711 KB vocabulary. */
export function getTokenizer(): Tokenizer {
  instance ??= new Tokenizer(JSON.parse(tokenizerJsonRaw), JSON.parse(tokenizerConfigRaw));
  return instance;
}

/** Model input length of `text`: [CLS] + WordPiece tokens + [SEP]. Never truncated. */
export function countModelTokens(text: string): number {
  return getTokenizer().encode(text, { add_special_tokens: true }).ids.length;
}
```

The model has its own vocabulary and its own way of splitting words into pieces. The only honest way to know whether a passage fits is to count with that same tokenizer. The two JSON files are copied verbatim from the Hugging Face model repository (`tokenizer/bge-small-en-v1.5/NOTICE.md` records the revision and the SHA-256 of each file) and bundled into the Worker with Vite's `?raw` import. The tokenizer is built once per isolate.

`passages.token_count` is this count, special tokens included, which is why a passage of 256 tokens fits the 512 limit with room to spare, and why the Python prototype's tiktoken count was replaced. Changing the files means bumping `TRANSFORM_VERSION` so every thread is re-transformed.

6. Track C: Vectorization, from GitHub text to a vector search

Vectorization is the whole path from a GitHub comment to a row in `passage_embeddings` that a query can hit. The path has stages, and each stage has a file. This track also reads the SQL that stores and searches the vectors, because half of the design lives there.

C.1 The job chain

worker/src/queue/pipeline.ts, lines 19 to 27

```
/** The job that follows a run of `kind`: every stream run feeds the transform, which feeds the
embedding. */
const NEXT_JOB_KIND: Record<string, string> = {
  ...Object.fromEntries(Object.keys(STREAM_KINDS).map((kind) => [kind, TRANSFORM_JOB_KIND])),
  [TRANSFORM_JOB_KIND]: EMBED_JOB_KIND,
};

export function nextJobKind(kind: string): string | undefined { return NEXT_JOB_KIND[kind]; }
```

Every `fetch_*`, `backfill_*` or `rescan_*` run that changed something queues `transform_passages`; every transform that touched something queues `embed_passages`. `ensureQueuedJob` (`jobs.ts:142–163`) makes sure only one job of a kind is active per repository at a time, so the chain is a pipeline, not a fan-out.

C.2 Extract

worker/src/etl/extract.ts, lines 15 to 29

```
export const HEAD_PAGES_PER_RUN = 2;
export const BACKFILL_PAGES_PER_RUN = 3;
export const BACKFILL_DELAY_SECONDS = 10;
export const MAX_PAGES_PER_RUN = 5;
export const PER_PAGE = 100;
export const MAX_ITEMS_PER_STREAM = 5000;
export const SYNC_OVERLAP_SECONDS = 300;
export const RATE_LIMIT_RESERVE = 20;
```

Three streams list GitHub endpoints 100 items per page: issues (which also returns pull requests), issue comments and review comments. A `StreamSpec` (line 35) says how to build the list URL and how to turn an item into a `sources` row. The rows keep the body as GitHub sent it, the author, the creation time and the `html_url` that becomes the permalink. Each stream has a durable checkpoint with a high-water mark for syncs and a cursor for the backfill. Extraction is not the subject of this track, but it defines the inputs: one `sources` row per opening post or comment, keyed by repository id, type and GitHub id.

C.3 Normalize

worker/src/etl/normalize.ts, lines 93 to 106

```
export function normalizeMarkdown(text: string | null | undefined): string {
  if (!text) return "";
  let result = text.replace(/\r\n?/g, "\n");
  const [protectedText, saved] = protectCode(result);
  result = protectedText;
  for (const [pattern, replacement] of RULES) {
    result = typeof replacement === "string" ? result.replace(pattern, replacement) :
result.replace(pattern, replacement);
  }
  result = decodeHTML(result);
  result = result.replace(/[ \t]+$/gm, "");
  result = result.replace(/\n{3,}/g, "\n\n");
  result = result.replace(PLACEHOLDER_RE, (_match, index: string) => saved[Number(index)]);
  return result.trim();
}
```

An embedding model reads prose better than markup. The normalizer removes HTML comments and tags, turns images into `[image: alt]`, turns links into `label (url)`, strips heading markers' trailing hashes, bold, strikethrough and horizontal rules, and standardizes bullets. Code is never changed: fenced blocks and inline code spans are swapped for a sentinel before the rules run and restored verbatim afterwards (lines 62 to 91). Underscore emphasis is left alone so `__init__` survives. The output kind is recorded on every passage as `normalized_markdown`. The module is a port of `pipeline/markdown_normalize.py`; both are checked against the same fixtures in `tests/fixtures`.

C.4 Chunk

worker/src/etl/chunk.ts, lines 14 and 132 to 144

```
/** Model tokens per passage, [CLS] and [SEP] included: one to three paragraphs of prose. */
export const MAX_PASSAGE_TOKENS = 256;
…
const LEVELS: Array<[(text: string) => string[], string]> = [
  [splitParagraphs, "\n\n"],
  [(text) => text.split("\n"), "\n"],
  [(text) => text.split(SENTENCE_END_RE), " "],
  [(text) => text.split(" "), " "],
];

/** Splits `text` into chunks of at most `maxTokens`, preferring large boundaries. */
export function splitText(text: string, maxTokens: number, countTokens: TokenCounter): string[] {
  return split(text, counting(maxTokens, countTokens), 0).map((chunk) => chunk.trim()).filter(Boolean);
}
```

A unit that fits in 256 tokens is one passage. A unit that does not is split at the largest boundary that works: paragraphs first, then lines, sentences, words, and as a last resort characters (`hardSplit`, line 199, for a long URL). Chunks are packed greedily. Three rules keep the chunks readable: a fenced code block is one paragraph and an oversized block is split into smaller blocks that each keep both fence lines; a heading, a short label line or a line ending with a colon moves to the next chunk together with the paragraph it introduces; a passage never mixes two speakers, because splitting happens per unit.

worker/src/etl/chunk.ts, lines 73 to 97: the tokenize-once trick

```
type Counting = { maxTokens: number; countTokens: TokenCounter; overhead: Map<string, number> };

function overheadOf(ctx: Counting, separator: string): number {
  let value = ctx.overhead.get(separator);
  if (value === undefined) {
    value = ctx.countTokens("a") + ctx.countTokens("b") - ctx.countTokens(`a${separator}b`);
    ctx.overhead.set(separator, value);
  }
  return value;
}

/** Tokens of `counts.length` parts joined by `separator`, from the parts' own counts. */
function joined(ctx: Counting, counts: number[], separator: string): number {
  let sum = 0;
  for (const c of counts) sum += c;
  return sum - overheadOf(ctx, separator) * (counts.length - 1);
}
```

Tokenizing is the Worker's only CPU-heavy work (A.6). The naive splitter re-tokenized the growing chunk after every part. This version tokenizes each part once and computes the count of a join from the parts' counts: for WordPiece with `[CLS]` and `[SEP]`, joining two parts with whitespace costs the sum of their counts minus 2, because a whitespace separator never merges or splits a token. The overhead is probed per separator, so a character counter gets its separator length back. The comment at line 71 records the proof: 3,984 joins of real GitHub text, 0 mismatches.

C.5 Passages, deterministic ids and content hashes

worker/src/etl/passages.ts, lines 29 to 42 and 186 to 190

```
/** github:{repository_id}:{type}:{number}; the numeric repository id survives renames. */
export function documentId(repositoryId: number, type: DocumentType, number: number): string {
  return `github:${repositoryId}:${type}:${number}`;
}

/** {document_id}:body:{i} for the opening, {document_id}:{source_type}:{github_id}:{i} for a comment. */
export function passageId(documentId: string, unit: UnitInput, index: number): string {
  const segment = isOpening(unit) ? "body" : `${unit.source_type}:${unit.source_github_id}`;
  return `${documentId}:${segment}:${index}`;
}
…
/** SHA-256 of the UTF-8 bytes of `text`, lowercase hex. */
export async function contentHash(text: string): Promise<string> {
  const digest = await crypto.subtle.digest("SHA-256", new TextEncoder().encode(text));
  return Array.from(new Uint8Array(digest), (b) => b.toString(16).padStart(2, "0")).join("");
}
```

ADR-0004 is the rule that makes the whole pipeline idempotent. A document is a thread: `github:71542716:issue:86`. A passage is one chunk of one unit: `github:71542716:issue:86:issue_comment:2242853511:0`. The source type is in the id because issue-comment and review-comment ids are separate GitHub sequences and can collide. The content hash is the SHA-256 of the passage text. Together they give three properties: a re-run writes nothing for unchanged text; a changed comment replaces its own passage and its own vector; and an evaluation label can name a passage that still exists months later. `buildPassages` (lines 140 to 181) walks the opening body and then each comment in thread order, normalizes, splits and labels each chunk with a locator such as `Comment 2 of 5 (part 1 of 3)` or `Review comment 4 of 4 on index.d.ts:24`.

C.6 Writing passages: sync_passages and two-phase indexing

migrations/20261013164333_passage_index_state.sql, lines 1 to 13 and the `on conflict` clause of `sync_passages`

```
-- Two-phase indexing. A passage is written as pending_index by the transform, its vector is
-- stored, and only then is it marked active, in the same transaction as the vector. Search
-- returns active passages only, so a passage whose vector failed to store, or whose text
-- changed after it was embedded, is never served from a stale or missing vector.
alter table public.passages
  add column index_state text not null default 'pending_index'
  check (index_state in ('pending_index', 'active'));
…
    on conflict (passage_id) do update set
      …
      content_hash      = excluded.content_hash,
      index_state       = case when p.content_hash is distinct from excluded.content_hash
                               then 'pending_index' else p.index_state end,
      updated_at        = now()
    where (p.content_hash, p.chunk_count, …) is distinct from (excluded.content_hash,
excluded.chunk_count, …)
```

The transform sends its documents and passages as two JSON arrays to one SQL function, `sync_passages`. In one transaction it upserts the documents, upserts the passages whose fields changed (an unchanged row is not even touched, so its vector stays active), resets `index_state` to `pending_index` when the text changed, and deletes passages of those documents that are no longer in the list. A deleted passage cascades to its vector. The counts come back as `{ documents, inserted, updated, unchanged, removed }` and land in the job result.

C.7 Storing vectors: the pgvector table and the HNSW index

migrations/20261013161046_passage_embeddings.sql, lines 1 to 23

```
create extension if not exists vector with schema extensions;

create table public.passage_embeddings (
  passage_id      text        primary key references public.passages (passage_id) on delete cascade,
  repository_id   bigint      not null references public.repositories (id) on delete cascade,
  model           text        not null,
  pooling         text        not null,
  dims            integer     not null,
  content_hash    text        not null,
  token_count     integer     not null,
  embedding       extensions.vector(384) not null,
  embedded_by_job bigint      references public.jobs (id) on delete set null,
  created_at      timestamptz not null default now(),
  updated_at      timestamptz not null default now(),
  constraint passage_embeddings_pinned_model
    check (model = '@cf/baai/bge-small-en-v1.5' and pooling = 'cls' and dims = 384)
);

create index passage_embeddings_repository_idx on public.passage_embeddings (repository_id);
create index passage_embeddings_embedding_idx
  on public.passage_embeddings using hnsw (embedding extensions.vector_cosine_ops);
```

Three things to learn here.

- **The `vector(384)` type** comes from the pgvector extension. A value is written as a JSON-like array text, `[0.01, -0.2, …]`, and stored as 384 float4 values, about 1.5 kB.
- **The CHECK constraint** pins model, pooling and dimensions. No code path can store a vector from another model, and the stored `model` column lets a future migration relax the constraint and re-embed without changing ids.
- **The HNSW index** with `vector_cosine_ops` is the approximate nearest-neighbour structure that makes a cosine search fast. It is built on insert, so there is no separate indexing step and no need to wait for enough rows, unlike an IVFFlat index.

migrations/20261013164333_passage_index_state.sql, `upsert_passage_embeddings` (the core)

```
insert into public.passage_embeddings as e
  (passage_id, repository_id, model, pooling, dims, content_hash, token_count, embedding,
embedded_by_job)
select passage_id, repository_id, model, pooling, dims, content_hash, token_count,
       (embedding::text)::extensions.vector(384), embedded_by_job
  from incoming
on conflict (passage_id) do update set
  model = excluded.model, pooling = excluded.pooling, dims = excluded.dims,
  content_hash = excluded.content_hash, token_count = excluded.token_count,
  embedding = excluded.embedding, embedded_by_job = excluded.embedded_by_job, updated_at = now()
returning (xmax = 0) as was_inserted
…
update public.passages p
   set index_state = 'active'
  from jsonb_to_recordset(p_rows) as r(passage_id text, content_hash text)
 where p.passage_id = r.passage_id
   and p.content_hash = r.content_hash;
```

The Worker sends the batch as one JSON array; `jsonb_to_recordset` unpacks it, and `(embedding::text)::vector(384)` turns the JSON array into a vector, because a JSON array's text form is valid pgvector input. The same transaction marks the passage active, but only when its current hash still matches the hash that was embedded. A passage whose text changed between the transform and the embed stays pending and is picked up by the next run. `pending_passages` is the mirror image: a passage with no vector row, or whose hash

differs, or whose model, pooling or dims differ. The two later migrations (`20261016131500`, `20261016133000`) change only the plan: with 70,000 passages a generic plan took 6 to 8 seconds, so the functions became plpgsql with `plan_cache_mode = force_custom_plan`.

C.8 Searching vectors: cosine on the HNSW index

`migrations/20261015100617_clamp_search_limit.sql`, the current `search_passages`

```
create or replace function public.search_passages(p_repository_id bigint, p_query extensions.vector(384),
p_limit integer)
returns table (passage_id text, document_id text, score real)
language plpgsql stable security invoker set search_path = ''
as $$
begin
  perform set_config('hnsw.ef_search', '200', true);
  perform set_config('hnsw.iterative_scan', 'relaxed_order', true);
  return query
    select e.passage_id, p.document_id, (1 - (e.embedding operator(extensions.<=>) p_query))::real as
score
      from public.passage_embeddings e
      join public.passages p on p.passage_id = e.passage_id
     where e.repository_id = p_repository_id
       and p.index_state = 'active'
     order by e.embedding operator(extensions.<=>) p_query
     limit least(greatest(coalesce(p_limit, 1), 1), 10);
end;
$$;
```

- `<=>` is pgvector's cosine distance operator. Distance 0 means identical, 2 means opposite. The score is `1 - distance`, so 1 means identical. `order by … <=> p_query` is what the HNSW index serves.
- `hnsw.ef_search = 200` widens the candidate list the index walks (the default is 40), which raises recall at a small cost in time.
- `hnsw.iterative_scan = relaxed_order` solves the filtered-search problem: the index returns candidates, the `where` clause drops the other repositories' rows, and the scan keeps pulling candidates until `p_limit` rows pass. Without it a filtered query can come back short.
- The limit is clamped to 10 inside the function, so no caller can ask for more than the cards ever show.
- The operator is schema-qualified because `search_path` is empty inside the function, a Supabase security convention.

`worker/src/api/search.ts`, lines 226 to 246

```
export async function searchSemantic(db, repositoryId, queryVector: number[],
  limit = SEARCH_LIMIT_DEFAULT): Promise<SearchMatch[]> {
  if (!Number.isSafeInteger(repositoryId) || repositoryId <= 0)
    throw new Error(`invalid repository id ${repositoryId}`);
  const { data, error } = await db.rpc("search_passages", {
    p_repository_id: repositoryId,
    p_query: JSON.stringify(queryVector),
    p_limit: Math.min(Math.max(limit, 1), SEARCH_LIMIT_MAX),
  });
  if (error) throw dbError("search_passages", error);
  return ownRows((data ?? []) as SearchMatch[], repositoryId, "search_passages");
}

function ownRows(matches: SearchMatch[], repositoryId: number, source: string): SearchMatch[] {
  const own = matches.filter((m) => belongsToRepository(m.document_id, repositoryId));
  if (own.length !== matches.length)
    console.error(`${source} returned ${matches.length - own.length} rows outside repository
${repositoryId}`);
  return own;
}
```

The Worker passes the query vector as a JSON string; PostgREST casts it to `vector(384)`. The result is checked once more in JavaScript: every document id embeds the repository id, so a row from another repository is dropped and logged even if the SQL were ever wrong. `tests/integration/search.test.ts` seeds the same four texts in two repositories and asserts that a search in one never returns the other.

C.9 The keyword baseline

`migrations/20261013170005_keyword_search.sql`, lines 10 to 36

```
create index passages_text_fts_idx on public.passages
  using gin (pg_catalog.to_tsvector('pg_catalog.english'::pg_catalog.regconfig, text));

create or replace function public.search_passages_keyword(p_repository_id bigint, p_query text, p_limit
integer)
returns table (passage_id text, document_id text, score real) … as $$
  with q as (
    select pg_catalog.websearch_to_tsquery('pg_catalog.english'::pg_catalog.regconfig, p_query) as tsq
  )
  select p.passage_id, p.document_id,
         pg_catalog.ts_rank_cd(pg_catalog.to_tsvector('pg_catalog.english'::pg_catalog.regconfig,
p.text), q.tsq)::real as score
    from public.passages p, q
   where p.repository_id = p_repository_id
     and pg_catalog.numnode(q.tsq) > 0
     and pg_catalog.to_tsvector('pg_catalog.english'::pg_catalog.regconfig, p.text) @@ q.tsq
   order by score desc, p.passage_id
   limit greatest(1, least(p_limit, 50));
$$;
```

Postgres full-text search needs no vector, so a passage still waiting for its embedding is already findable by the words it contains, and the service keeps answering when Workers AI is paused. The GIN index stores the lexemes of every passage; `websearch_to_tsquery` parses quotes and `-word`; `ts_rank_cd` ranks by cover density. The function mirrors `search_passages` in arguments and columns so the API treats both the same. The evaluation (C.11) showed the limit of this baseline for questions: `websearch_to_tsquery` ANDs every word, so a 12-word question rarely matches a passage.

C.10 Deciding what to show

`worker/src/api/search.ts`, lines 16 to 23 and 106 to 133

```
export const SCORE_HIGH = 0.8;
export const SCORE_LOW = 0.7;
…
  } else if (top >= SCORE_HIGH) {
    [mode, confidence, message, hits] = ["semantic", "high", null, semantic.hits];
  } else {
    const keyword = await searchKeyword(ctx.db, repositoryId, query, limit);
    const building = index.embedded === 0 && index.pending > 0;
    if (keyword.length > 0) {
      [mode, confidence, hits] = ["keyword", "low", keyword];
      message = `${
        unavailable ? unavailable
          : building ? `The semantic index is still building (...)`
          : "No discussion is a close semantic match."
      } Showing passages that contain your search words; verify them at the source.`;
    } else if (top >= SCORE_LOW) {
      [mode, confidence, hits] = ["semantic", "low", semantic.hits];
      message = "Only weak matches were found. This repository may not discuss the topic; " +
        "verify them at the source before relying on them.";
    } else {
      [mode, confidence, hits] = ["none", "none", []];
      message = … "No supporting discussion was found for this query in this repository.";
    }
  }
```

In `auto` mode the semantic search runs first. If the top cosine score is at or above 0.80 the semantic results stand alone with `high` confidence. Otherwise the keyword baseline runs; if it matches, its results are shown as `low` with a message that says why. If it does not match and the top score is at least 0.70, the weak semantic results are shown with a warning. Below that the answer is "no supporting discussion". The thresholds were calibrated on p-limit on 2026-10-13: questions the repository answers scored 0.77 to 0.89, an exact identifier 0.77, off-topic questions at most 0.67 (ADR-0006). The forced modes `semantic` and `keyword` run one retrieval alone, for the evaluation.

Whatever the mode, the hits become evidence cards through `resolveEvidence` (`api/evidence.ts:67–114`). The card carries the thread title, the locator, the author, the date, a 300-character excerpt and the permalink. `verifySourceLink` (lines 28 to 47) checks that the link is an `https://github.com` URL to the right thread with the right anchor (`#issuecomment-{id}` or `#discussion_r{id}`) before the card is shown; a hit that fails is left out and counted in `unverified`. The response always carries the index state, so a partial or capped index says so (lines 153 to 179).

C.11 What the evaluation showed

`docs/EVALUATION.md` reports a frozen set of 25 questions over four small repositories (p-queue, p-limit, yocto-queue, p-defer), 20 answerable and 5 not, measured against the live service on 2026-10-15 through the public API only.

Mode	Recall@3	Hit@3	MRR@5	Zero hits in top 5 (of 20)	Thread Hit@5
semantic	0.292	0.65	0.546	6	0.95
keyword	0.05	0.05	0.05	19	0.10
auto	0.292	0.65	0.546	6	0.95

Lessons worth carrying into your own embedding projects:

- **The right thread is easy, the right comment is hard.** Five of the seven semantic misses returned the right issue or pull request and a different comment. Grouping results by thread or boosting sibling units is the first improvement listed in `LATER.md`.
- **Quality falls with corpus size.** p-queue, with 1,403 passages and eight years of recurring topics, scored 0.083 Recall@3; p-defer, with 24 passages, scored 0.5.
- **The top score does not separate supported from unsupported questions.** Supported questions' top scores ran from 0.805 to 0.910, unsupported ones from 0.702 to 0.813. A threshold on the top score alone cannot abstain reliably; a score gap or a margin between the first and fifth result is the next candidate.
- **AND-ed keyword search is a floor, not a competitor.** It returned nothing for 17 of 20 questions and worked only for rare exact words such as "CommonJS", "require" and "Jest".
- **Freeze the question set before you tune.** The set's SHA-256 is pinned by a test, and the report discloses that the thresholds were calibrated on one of the evaluation repositories.

7. Reading paths and exercises

Path A: Cloudflare Workers, about two hours

1. `docs/ARCHITECTURE.md` and `docs/DECISIONS.md` (ADR-0002, ADR-0008, ADR-0009).
2. `wrangler.jsonc`, then `worker-configuration.d.ts` lines 1 to 30.
3. `worker/src/index.ts`: the handlers (42 to 94), then `route` (96 to 145), then `searchRepository` (296 to 312).
4. `worker/src/queue/jobs.ts`, then `consumer.ts`, then `retry.ts` for the backoff and the error classes.
5. `worker/src/queue/budget.ts` and `migrations/20261014164438_usage_budgets.sql`.
6. `worker/src/cron/sync.ts` and `reconcile.ts`.
7. `vite.config.ts`, `vitest.config.ts`, `tests/integration/consumer.test.ts`, `docs/milestone-7-acceptance.md`.

Path B: Embeddings, about ninety minutes

1. ADR-0003 in `docs/DECISIONS.md`.
2. `worker/src/etl/embeddings.ts`, top to bottom. Then `tests/integration/embeddings.test.ts` side by side.
3. `worker/src/etl/tokenizer.ts` and `tokenizer/bge-small-en-v1.5/NOTICE.md`.
4. `worker/src/etl/embed.ts`, then `tests/integration/embed-passages.test.ts`.
5. `worker/src/api/search.ts` lines 44 to 63 and 182 to 197.
6. `docs/benchmarks/ct-24-transform-embed.md` for the metering and the failure drills.

Path C: Vectorization, about two hours

1. ADR-0001, ADR-0004 and ADR-0006.
2. `pipeline/README.md`, then `pipeline/chunk_passages.py`'s docstring: the specification in prose.
3. `worker/src/etl/normalize.ts`, `chunk.ts`, `passages.ts`, `transform.ts`, with `tests/integration/chunk.test.ts` and `passages.test.ts`.
4. Migrations in order: `20261013150724_documents_and_passages`, `20261013151622_sync_passages`, `20261013161046_passage_embeddings`, `20261013164050_search_passages`, `20261013164333_passage_index_state`, `20261013170005_keyword_search`, `20261015100617_clamp_search_limit`, `20261016133000_custom_plans`.
5. `worker/src/api/search.ts` and `evidence.ts`, with `tests/integration/search.test.ts` and `search-api.test.ts`.
6. `docs/EVALUATION.md`, `tests/evaluation/metrics.ts` and `run.ts`.

Exercises

1. **Run the chain locally.** `nvm use 22 && npm run dev`, then register a small public repository with `POST /api/repositories`. Watch the dev server log: head pass, transform, embed, and the `durationMs` of each job. Read the `jobs` rows in Supabase and match them to the states in A.4.
2. **Trigger one job by hand.** With `ADMIN_TOKEN` set, call `POST /api/repositories/:id/jobs` with `{"kind":"embed_passages"}` and then `GET /api/repositories/:id/embeddings/stats`. Compare `tokens` with `billed_tokens` and compute the neurons by hand with B.5.
3. **Measure the longest-text rule.** Set `EMBEDDING_BATCH_SIZE` to 25 in a local branch, delete the repository's rows in `passage_embeddings`, re-run the embed job and compare the padding overhead with the 27% in CT-24. Put the constant back.

4. **Compare the retrievals.** Send the same question to `POST /api/search` with `mode: "semantic"`, `"keyword"` and `"auto"`. Explain each response's `mode`, `confidence` and `message` from C.10.
5. **Query pgvector directly.** In the SQL editor, take any stored vector and run the `select … order by embedding <=> '[…]' limit 5` from C.8 by hand, once with and once without `set hnsw.iterative_scan = 'relaxed_order'`, with a repository filter. Count the rows.
6. **Break a vector on purpose.** In `tests/integration/embed-passages.test.ts`, find the test that injects a NaN vector through `FakeEmbeddingClient`. Predict the counts in `EmbedResult` before you read the assertions.
7. **Run the live model test.** With `CLOUDFLARE_ACCOUNT_ID` and `WORKERS_AI_API_TOKEN` in `.env`, run `npx vitest run tests/integration/embeddings.test.ts` and read the logged metered tokens.
8. **Design exercise.** List every place that would change to add a second embedding model next to the first: the CHECK constraint, the predicates in `pending_passages`, the `EMBEDDING_*` constants, `TRANSFORM_VERSION`, the query embedding, the evaluation. `LATER.md` has one line on it; write the plan.

8. Glossary and references

Glossary

Term	Meaning in this project
Binding	A resource attached to the Worker by `wrangler.jsonc` and reached through `env`: the queue, the rate limiters.
Budget	A row in `usage_budgets` at 80% of a free allowance, reserved atomically before every metered call.
cls pooling	Taking the model's output at the `[CLS]` token as the sentence vector. The native pooling of bge-small-en-v1.5.
Content hash	SHA-256 of a passage's text; decides whether a passage is re-embedded.
Cosine similarity	1 minus pgvector's `<=>` distance; 1 means identical direction.
Document	One thread: an issue or a pull request with its comments.
Evidence card	A search result: passage excerpt, locator, author, date and a verified permalink.
Generation	A counter on the job row, bumped on every enqueue and carried in the message, so stale messages are recognised.
HNSW	Hierarchical Navigable Small World, the graph index pgvector uses for approximate nearest-neighbour search.
Lease	A claim on a job by one consumer for 300 s; another consumer takes over only after it expires.
Neuron	Workers AI's unit of metering; 1,841 per million input tokens for this model, 10,000 per day free.
Passage	One chunk of one unit, at most 256 model tokens, with a deterministic id.
Source	One stored GitHub record: an issue, a pull request, an issue comment or a review comment.
Stream	One GitHub list endpoint with its own checkpoint: issues, issue comments, review comments.
Unit	The opening body of a thread or one comment; a passage never mixes two units.
WordPiece	The sub-word tokenization of BERT-style models; the vendored `tokenizer.json` defines it.

External references

- Cloudflare Workers: developers.cloudflare.com/workers (configuration, handlers, bindings, limits).
- Cloudflare Queues: developers.cloudflare.com/queues (producers, consumers, batching, delayed delivery, retries).
- Cloudflare Workers AI, text embeddings and the model page for `@cf/baai/bge-small-en-v1.5`: developers.cloudflare.com/workers-ai.
- Workers AI REST endpoint: `POST /accounts/{account_id}/ai/run/{model}` on api.cloudflare.com/client/v4.
- Workers AI pricing and neurons: developers.cloudflare.com/workers-ai/platform/pricing.
- pgvector: github.com/pgvector/pgvector (vector type, `<=>`, HNSW, `ef_search`, iterative scans).
- Supabase vector columns and RPC: supabase.com/docs/guides/ai.
- Postgres full-text search: postgresql.org/docs/current/textsearch.html.
- The model card: huggingface.co/BAAI/bge-small-en-v1.5.
- Hugging Face tokenizers for JavaScript: github.com/huggingface/tokenizers.
- Cloudflare Vite plugin and Vitest integration: developers.cloudflare.com/workers/vite-plugin and developers.cloudflare.com/workers/testing/vitest-integration.

Appendix: File index

File	What to learn there	Track
`wrangler.jsonc`	Every binding, the cron, the queue, the rate limiters, the assets rule.	A
`worker-configuration.d.ts`	The generated `Env` type; which names are bindings, vars or secrets.	A
`worker/src/index.ts`	The three handlers, the router, request validation, error responses, CORS.	A
`worker/src/queue/jobs.ts`	Message shape, metered sends, enqueue with generation, claim with lease, finish fenced by owner.	A
`worker/src/queue/consumer.ts`	ack and retry, continuation jobs, follow-ups, transient versus permanent failures, budget pauses.	A
`worker/src/queue/retry.ts`	`JobError`, error classification, exponential backoff, `MAX_ATTEMPTS`.	A, B
`worker/src/queue/budget.ts`	Reserve, take, settle, gauge; milli-neurons; what a refused reservation raises.	A, B
`worker/src/cron/*.ts`	Reconcile, sync and rescan scheduling, maintenance and retention.	A
`worker/src/api/ratelimit.ts`, `cors.ts`, `turnstile.ts`, `admin.ts`	The protections in front of the routes.	A
`worker/src/etl/embeddings.ts`	The Workers AI REST client, constants, error mapping, metering helpers, vector validation, the fake client.	B
`worker/src/etl/embed.ts`	The embed job: pending passages, partition, sort, batch, reserve, store, usage.	B
`worker/src/etl/tokenizer.ts`	Counting model tokens with the vendored WordPiece tokenizer.	B, C
`worker/src/etl/extract.ts`, `github-rest.ts`, `sources.ts`, `checkpoints.ts`	Streams, pages, cursors, high-water marks, source rows.	C
`worker/src/etl/normalize.ts`	Markdown to plain text with code protected.	C
`worker/src/etl/chunk.ts`	Boundary-aware splitting under a token budget; tokenize-once counting.	C
`worker/src/etl/passages.ts`	Document and passage ids, locators, content hashes, the 20,000-character cap.	C
`worker/src/etl/transform.ts`, `documents.ts`	The transform job and its database calls.	C
`worker/src/api/search.ts`	Query embedding, semantic and keyword retrieval, thresholds, the auto decision, index state.	B, C
`worker/src/api/evidence.ts`	Permalink verification and evidence cards.	C
`shared/api.ts`	The API contract shared by the Worker and the SPA: `SearchResponse`, `EvidenceCard`, `EmbeddingStats`.	all
`migrations/*.sql`	Tables, RLS, grants and the SQL functions: `sync_passages`, `pending_passages`, `upsert_passage_embeddings`, `search_passages`, `search_passages_keyword`, `reserve_budget`.	C
`tests/integration/*.test.ts`	How each module is exercised in workerd against the live database.	all
`tests/evaluation/`	The frozen question set, the metrics and the runner.	C
`docs/`	Architecture, decisions, benchmarks, milestone acceptance, evaluation.	all
`pipeline/`	The Python prototype: the specification the Worker ports.	C